Plano e Relatório De Testes - Compass ServeRest

1 - Plano de Testes

1.1 - Objeto de Testes

O objeto de testes é a API REST da compass: https://compassuol.serverest.dev/ .

1.2 - Escopo

Os testes abrangerão prioritariamente os recursos de criação de carrinhos e cancelamento de compras, os métodos POST em /carrinhos e DELETE em /carrinhos/cancelar-compra respectivamente. Aproveitando a necessidade, os métodos POST /login e POST /produtos também serão testados.

1.3 - Objetivos

- Automatizar os testes descritos no escopo;
- Verificar se as requisições que gerenciam os objetos “produtos” e “carrinhos” manipulam corretamente os dados e retornam as respostas corretas.
- Verificar a funcionalidade da autenticação e permissões;
- Analisar se a lógica de negócios dos endpoints está sendo devidamente aplicada.

1.4 - Riscos e Prioridades

Principais riscos identificados na API levando em consideração o objetivo prioritário dos testes na criação de carrinhos e cancelamento de compras:

- O usuário não conseguir efetuar o login ou não gerar token de autenticação;
- O usuário conseguir acessar um carrinho sem a devida autenticação;
- Impossibilidade de adicionar produtos ao carrinho;
- Valores incorretos serem salvos no estoque;
- O usuário adquirir produtos fora de estoque.

1.5 - Cenários de Teste para automação

ID do Cenário	Endpoint	Verbo HTTP	Cenário	Resultado Esperado
TL-01	/login	POST	Realizar um login com dados válidos e usuário cadastrado.	A aplicação deve permitir o acesso e retornar o token.
TL-02	/login	POST	Realizar login com dados inválidos.	A aplicação deve retornar um status code 400 e uma mensagem indicando o erro.
TL-03	/login	POST	Realizar login com um email não cadastrado.	A aplicação deve retornar um status code 401 e uma mensagem indicando o erro.
TL-04	/login	POST	Realizar um login com email válido e senha incorreta.	A aplicação deve retornar um status code 401 e uma

				mensagem indicando o erro.
TP-01	/produtos	POST	Cadastrar um produto com dados válidos.	Cadastrar o produto e exibir uma mensagem de sucesso.
TP-02	/produtos	POST	Cadastrar um produto com dados inválidos.	Deverá ser retornado um status code 400 e uma mensagem indicando o erro.
TP-03	/produtos	POST	Cadastrar um produto com um nome já utilizado.	Deverá ser retornado um status code 400 e uma mensagem indicando o erro.
TP-04	/produtos	POST	Cadastrar um produto sem um usuário administrador autenticado.	Deverá ser retornado um status code 401 e uma mensagem indicando o erro.
TC-01	/carrinhos	POST	Cadastrar um novo carrinho com dados válidos e token válido.	Cadastrar o carrinho, exibir uma mensagem de sucesso e subtrair a quantidade de produtos inserida dos respectivos objetos em /produtos.
TC-02	/carrinhos	POST	Cadastrar um novo carrinho sem um token válido.	Deve ser retornado um status code 401 e uma mensagem de erro de autorização.
TC-03	/carrinhos	POST	Cadastrar um novo carrinho com mais produtos que em estoque.	Deve ser retornado um status code 400 e uma mensagem de erro correspondente.
TC-04	/carrinhos	POST	Cadastrar um novo carrinho com um token que já possui um carrinho vinculado.	Retornar uma mensagem de erro informando que o usuário já possui um carrinho cadastrado.
TC-05	/carrinhos/cancelar-compra	DELETE	Cancelar uma compra com um token válido.	O carrinho deve ser excluído, a quantidade de cada produto deve ser inserida de volta respectivamente em cada objeto de /produtos.
TC-06	/carrinhos/cancelar-compra	DELETE	Cancelar uma compra sem um token válido.	Deve ser retornado um status 401 e uma mensagem de erro de autorização.
TC-07	/carrinhos/cancelar-compra	DELETE	Cancelar uma compra utilizando um token que não possui um carrinho vinculado.	Deverá ser retornado um status code 400 e uma mensagem indicando o erro.

1.6 - Critérios de Entrada

Para o início dos testes é necessário que as funcionalidades exigidas para a compra de produtos estejam todas desenvolvidas, sendo elas: a criação de produtos, a criação de usuários (ou pelo menos um usuário válido criado) e a função de login (que também é crítica para a aplicação).

1.7 - Critérios de Saída

Para que esta etapa de testes seja considerada concluída é necessário que os testes dos endpoints mais críticos (/login e /carrinhos) estejam automatizados e de preferência automatizar testes do endpoint /produtos para aproveitar as keywords que serão necessários para criar a massa de dados dos outros endpoints e garantir que estão sendo criados corretamente.

2 - Relatório de Execução

Projeto: Challenge - Compass API ServeRest - RobotFramework

Testador: Juan Castro Mendonça de Oliveira

Data: 19/03/2026

Ambiente: Windows 11 | Rede Wifi 5g

Browser: Microsoft Edge

IDE: PyCharm

2.1 - Execução - Cenários de Teste

ID do Cenário	Resultado Esperado	Resultado Obtido
TL-01	A aplicação deve permitir o acesso e retornar o token.	Conforme o esperado.
TL-02	A aplicação deve retornar um status code 400 e uma mensagem indicando o erro.	Conforme o esperado.
TL-03	A aplicação deve retornar um status code 401 e uma mensagem indicando o erro.	Conforme o esperado.
TL-04	A aplicação deve retornar um status code 401 e uma mensagem indicando o erro.	Conforme o esperado.
TP-01	Cadastrar o produto e exibir uma mensagem de sucesso.	Conforme o esperado.
TP-02	Deverá ser retornado um status code 400 e uma mensagem indicando o erro.	Conforme o esperado.
TP-03	Deverá ser retornado um status code 400 e uma mensagem indicando o erro.	Conforme o esperado.
TP-04	Deverá ser retornado um status code 401 e uma mensagem indicando o erro.	Conforme o esperado.

TC-01	Cadastrar o carrinho, exibir uma mensagem de sucesso e subtrair a quantidade de produtos inserida dos respectivos objetos em /produtos.	Conforme o esperado.
TC-02	Deve ser retornado um status code 401 e uma mensagem de erro de autorização.	Conforme o esperado.
TC-03	Deve ser retornado um status code 400 e uma mensagem de erro correspondente.	Conforme o esperado.
TC-04	Retornar uma mensagem de erro informando que o usuário já possui um carrinho cadastrado.	Conforme o esperado.
TC-05	O carrinho deve ser excluído, a quantidade de cada produto deve ser inserida de volta respectivamente em cada objeto de /produtos.	Falha no teste: A aplicação retorna a diferença e concatena com a quantidade em estoque, gerando um número extremamente maior.
TC-06	Deve ser retornado um status 401 e uma mensagem de erro de autorização.	Conforme o esperado.
TC-07	Deverá ser retornado um status code 400 e uma mensagem indicando o erro.	Falha no teste: A aplicação retorna um status code 200 ao invés de 400.

2.2 - Conclusão - Cenários de Teste

A aplicação se provou bem consistente durante a execução dos testes, retornando resultados esperados e coerente com a documentação. Porém foram identificados dois bugs que podem comprometer a integridade do estoque e a comunicação da aplicação.

Foram executados os 15 cenários de teste descritos no plano de testes, sendo que 13 passaram com sucesso e 2 falharam.

É recomendado que:

- A lógica de retorno dos produtos ao estoque após o cancelamento de uma compra seja revista;
- Alterar o status code em caso do token não ter carrinho vinculado de 200 (OK) para 400 (BAD REQUEST).

2.3 - Report de Issues

I- Ao cancelar uma compra com um token que não possui carrinho vinculado é retornado um status 200 de sucesso.

Passo a passo: Autentique com um usuário que não possui carrinho vinculado; Envie uma requisição para cancelar a compra com o token de autenticação com o token gerado; A aplicação retornará um status code 200 com a mensagem "Não foi encontrado carrinho para esse usuário".

Resultado Esperado: A aplicação deveria retornar um status code 400 (Bad Request).

Prioridade: Média.

Evidência:

```
REPORT
TEST TC-07 - Cancelar uma compra sem um carrinho existente
Full Name:        Carrinho.TC-07 - Cancelar uma compra sem um carrinho existente
Documentation:    Cancelar uma compra com um token sem carrinho vinculado
Tags:             cart, delete, invalid
Start / End / Elapsed:  20260319 10:32:17.308 / 20260319 10:32:18.870 / 00:00:01.562
Status:           FAIL
Message:          Url: https://compassuol.serverest.dev/carrinhos/cancelar-compra Expected status: 200 != 400
+ SETUP ServeRest. Fazer login com "fulano@qa.com" e garantir que não tenha carrinho já criado   00:00:01.028
- KEYWORD ServeRest. Cancelar Uma Compra Sem Carrinho Vinculado   00:00:00.530
  Start / End / Elapsed:  20260319 10:32:18.338 / 20260319 10:32:18.868 / 00:00:00.530
  + KEYWORD &{headers} = BuiltIn. Create Dictionary  Authorization=Bearer ${TOKEN}   00:00:00.001
  - KEYWORD RequestsLibrary. DELETE  url=${URL}/${CANCELAR}  headers=&{headers}  expected_status=400   00:00:00.528
    Documentation:  Sends a DELETE request.
    Start / End / Elapsed:  20260319 10:32:18.340 / 20260319 10:32:18.868 / 00:00:00.528
    10:32:18.863  INFO  DELETE Request : url=https://compassuol.serverest.dev/carrinhos/cancelar-compra
      path_url=/carrinhos/cancelar-compra
      headers={'User-Agent': 'python-requests/2.32.5', 'Accept-Encoding': 'gzip, deflate, zstd', 'Accept': '*/*', 'Connection': 'keep-alive',
    'Authorization': 'Bearer
    eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6ImZ1bGFub0BxYS5jb20iLCJwYXNzd29yZCI6InR1c3RlIiwiaWF0IjoxNzczOTI3MTQ4LCJleHAiOjE3NzM5Mjc3NDh9.UEFV3k0
    1zUlY0uWC2PMRlGewe8OFPIejRL7TdPkXgoQ', 'Content-Length': '0'}
      body=None
    10:32:18.863  INFO  DELETE Response : url=https://compassuol.serverest.dev/carrinhos/cancelar-compra
      status=200, reason=OK
      headers={'access-control-allow-origin': '*', 'x-dns-prefetch-control': 'off', 'x-frame-options': 'SAMEORIGIN', 'strict-transport-security': 'max-
    age=15552000; includeSubDomains', 'x-download-options': 'noopen', 'x-content-type-options': 'nosniff', 'x-xss-protection': '1; mode=block', 'content-
    type': 'application/json; charset=utf-8', 'x-cloud-trace-context': '526292878c44b18333a45497b6590d00', 'date': 'Thu, 19 Mar 2026 13:32:29 GMT',
    'server': 'Google Frontend', 'Content-Length': '68'}
      body={
          "message": "Não foi encontrado carrinho para esse usuário"
    }
    10:32:18.865  FAIL  Url: https://compassuol.serverest.dev/carrinhos/cancelar-compra Expected status: 200 != 400
```

II- Ao enviar uma requisição de cancelar compra pelo python, a aplicação concatena o valor do estoque com o valor para retornar.

Passo a passo: Esteja autenticado com um usuário válido; Tenha um carrinho criado ou crie um para vincular ao token; Verifique a quantidade dos produtos que foram inseridos no carrinho que restaram no estoque; Envie uma requisição para cancelar a compra; Verifique novamente o estoque dos produtos; A quantidade em estoque estará concatenada com a quantidade que deve ser retornada do carrinho respectivamente.

Resultado Esperado:

Prioridade: Alta.

Evidência:

O estoque do produto era de 95 depois da criação do carrinho e a quantidade inserida no carrinho era 10. Ao invés de atualizar para 105, a aplicação atualizou o estoque para 9510.

```
- TEST TC-05 - Cancelar uma compra com sucesso   00:00:03.647
  Full Name:        Carrinho.TC-05 - Cancelar uma compra com sucesso
  Documentation:    Cancelar uma compra, excluindo o carrinho e retornando os produtos ao estoque
  Tags:             cart, delete, valid
  Start / End / Elapsed:  20260319 10:32:10.531 / 20260319 10:32:14.178 / 00:00:03.647
  Status:           FAIL
  Message:          9510 != 105
  + SETUP ServeRest. Fazer login com "fulano@qa.com" e garantir que não tenha carrinho já criado   00:00:01.040
  + KEYWORD ServeRest. Cadastrar Novo Produto  500  eletrodomesticos  105   00:00:00.498
  + KEYWORD ServeRest. Cadastrar Novo Carrinho  BeeJh5lz3k6kSIzA  5  ${ID_PRODUTO}  10   00:00:00.561
  + KEYWORD ServeRest. Verificar a quantidade de produtos no estoque após criação do carrinho  ${ID_PRODUTO}   00:00:00.523
  + KEYWORD ServeRest. Cancelar uma compra   00:00:00.515
  - KEYWORD ServeRest. Verificar se os produtos retornaram ao estoque  ${ID_PRODUTO}   00:00:00.503
    Start / Elapsed:  20260319 10:32:13.674 / 20260319 10:32:14.177 / 00:00:00.503
    + KEYWORD ${produto_response} = RequestsLibrary. GET  url=${URL}/produtos/${id}  expected_status=200   00:00:00.500
    - KEYWORD BuiltIn. Should Be Equal  ${produto_response.json()["quantidade"]}  ${QUANTIDADE_PRODUTO}   00:00:00.002
      Documentation:  Fails if the given objects are unequal.
      Start / Elapsed:  20260319 10:32:14.174 / 20260319 10:32:14.176 / 00:00:00.002
      10:32:14.176  FAIL  9510 != 105
```